# Chapter 15

# Closure Properties of Decidable Languages

We will study the closure properties of decidable and Turing recognizable languages under some of the standard operations on languages.

## 15.1 Closure Properties of Decidable and Turing Recognizable Languages

1. **Union**

   Both decidable and Turing recognizable languages are closed under union.

   - For decidable languages the proof is easy. Suppose $L_1$ and $L_2$ are two decidable languages accepted by halting TMs $M_1$ and $M_2$ respectively. The machine for $L_1 \cup L_2$ is designed as follows:

     Given an input $x$, simulate $M_1$ on $x$. If $M_1$ accepts then *accept*, else simulate $M_2$ on $x$. If $M_2$ accepts then *accept* else *reject*.
   - Now suppose $L_1$ and $L_2$ are two Turing recognizable languages accepted by TMs $M_1$ and $M_2$ respectively. Since $L_1$ and $L_2$ are Turing recognizable languages, therefore for strings that do not belong to these languages, the corresponding machines may not even halt. The previous strategy will not work because we can have a scenario where $M_2$ accepts $x$ but $M_1$ loops forever.

     Here the trick is to simulate both $M_1$ and $M_2$ "simultaneously". In other words, we design a machine that executes one step of $M_1$, followed by one step of $M_2$, then again one step of $M_1$ and so on.

2. **Concatenation**

   Both decidable and Turing recognizable languages are closed under concatenation.

   I will give the proof for Turing recognizable languages. The proof for decidable languages is similar. Let $L_1$ and $L_2$ be two Turing recognizable languages. Given an input $w$, use nondeterminism and guess a partition $w$ (say $w = xy$). Now run the respective Turing machines of $L_1$ and $L_2$ on $x$ and $y$ respectively. If both accepts then *accept* else *reject*.

3. **Star**

Both decidable and Turing recognizable languages are closed under star operation.

This is also similar to concatenation. Nondeterministically first guess a number $k$, and then guess a $k$ partition of the given input. Now for each string in the partition, check whether it belongs to the original language.

4. **Intersection**

   Both decidable and Turing recognizable languages are closed under intersection.

   Run the TMs of both the languages on the given input. *accept* if and only if both the machines accept. In the case of intersection we can run the TMs of $L_1$ and $L_2$ one after the other (as opposed to union).

5. **Complementation**

   - Decidable languages are closed under complementation. To design a machine for the complement of a language $L$, we can simulate the machine for $L$ on an input. If it accepts then *accept* and vice versa.
   - Turing recognizable languages are not closed under complement. In fact, Theorem 23 better explains the situation.

**Theorem 23.** *A language $L$ is decidable if and only if both $L$ and $\overline{L}$ are Turing recognizable.*

*Proof.* If $L$ is decidable then it is Turing recognizable. Moreover since decidable languages are closed under complement, $\overline{L}$ is also Turing recognizable.
Suppose $L$ is Turing recognizable via a TM $M$ and $\overline{L}$ is Turing recognizable via a TM $M'$. Given an input $x$, simulate $x$ on both the machines $M$ and $M'$ simultaneously (similar to union). If $M$ accepts then *accept* and if $M'$ accepts then *reject*. Observe that since the string $x$ either belongs to $L$ or $\overline{L}$ therefore one of the two machines must accept $x$.

□

# Chapter 16

# Decidability Properties of Regular and CFLs

## 16.1 Decidability Properties of Regular Languages

We will look at some properties of regular languages that are decidable, such as whether a language has a certain string, or whether it is empty or not, and so on. The language is presented either via an automaton that accepts the language or a regular expression.

### 16.1.1 DFA Acceptance Problem

Consider the following language,

$$A_{DFA} = \{\langle D, w\rangle \mid D \text{ is a DFA that accepts the string } w\}.$$

Is $A_{DFA}$ decidable?
Here $\langle D, w\rangle$ denotes an encoding of the DFA $D$ and a string $w$. It can be any encoding scheme as long as it is consistent.
The answer to the above question is yes. It is easy to construct a TM that accepts the language $A_{DFA}$.

**Description of a halting TM that accepts $A_{DFA}$**

**Input:** $\langle D, w\rangle$ where $D$ is a DFA and $w$ is a string.

1. Simulate $D$ on $w$.
2. If it ends at an accept state then *accept* else *reject*.

**Note 10.** Observe that instead of a DFA, if a regular language is presented as an NFA or an RE, the corresponding language is still decidable. This is because we have seen algorithms to convert an NFA or an RE to a DFA (RE to NFA to DFA).

### 16.1.2 DFA Emptiness Problem

Consider the language,

$$E_{DFA} = \{\langle D\rangle \mid D \text{ is a DFA and } L(D) = \emptyset\}.$$

$E_{DFA}$ is decidable as well. This is because, $L(D) = \emptyset$ if and only if there is no path in the DFA from the start state to any accept state. We can use a graph traversal algorithm such as DFS or BFS to check this and answer accordingly.

### 16.1.3 DFA Equality Problem

Consider the language,

$$EQ_{DFA} = \{\langle D_1, D_2\rangle \mid D_1, D_2 \text{ are 2 DFAs and } L(D_1) = L(D_2)\}.$$

Of course we cannot simulate every possible string on both $D_1$ and $D_2$ and verify whether they accept the same language, since the number of strings is infinite.
The trick is to use some closure properties of regular languages.
Firstly, observe that

$$L(D_1) = L(D_2) \iff \left(L(D_1) \cap \overline{L(D_2)}\right) \cup \left(\overline{L(D_1)} \cap L(D_2)\right) = \emptyset.$$

Using closure properties, we can construct a DFA $D$ such that

$$L(D) = \left(L(D_1) \cap \overline{L(D_2)}\right) \cup \left(\overline{L(D_1)} \cap L(D_2)\right).$$

Now using the algorithm for $E_{DFA}$ we can check whether $L(D) = \emptyset$ or not, and hence whether $L(D_1) = L(D_2)$.

## 16.2 Decidability Properties of CFLs

We now discuss the similar decidability properties for CFLs. Since CFLs are defined by CFGs and PDAs, we will assume that the CFL is presented to us as a CFG or a PDA. Once again, it does not matter whether the CFL is presented using a CFG or a PDA, since we can convert one to the other.

### 16.2.1 CFG Acceptance Problem

Consider the language,

$$A_{CFG} = \{\langle G, w\rangle \mid G \text{ is a CFG that accepts the string } w\}.$$

Recall that $G$ accepts $w$ if $S \stackrel{*}{\Rightarrow} w$, where $S$ is the start variable of $G$. So one possibility is to cycle through every possible derivation from $w$ and se if it yields $w$. However for an arbitrary grammar we do not know a priori how many steps are there is the derivation and hence we would not know when to stop. Fortunately Chomsky Normal Form comes to our rescue in this case. We know that every CFG can be converted to a CFG in Chomsky Normal Form. Secondly, for a grammar $G$ in Chomsky Normal Form, the number of steps in the derivation of a non-empty string $w \in L(G)$ is $2|w| - 1$ and is 1 for $\epsilon$. I leave this as an exercise.

**Exercise 30.** Let $G$ be a grammar in Chomsky Normal Form and $w \in L(G)$. Show that if $w$ is not the empty string then the length of any derivation of $w$ in $G$ is $2|w| - 1$ and if $w = \epsilon$ then the length of any derivation is 1.

**Description of a halting TM that accepts $A_{CFG}$**

**Input:** $\langle G, w\rangle$ where $G$ is a CFG that accepts the string $w$.

1. Convert $G$ to an equivalent CNF grammar, say $G'$.
2. List all derivations of length $2|w| - 1$ if $w \neq \epsilon$ and all derivations of length 1 if $w = \epsilon$.
3. If one of the derivations generate $w$ then *accept* else *reject*.

### 16.2.2 CFG Emptiness Problem

Consider the language,

$$E_{CFG} = \{\langle G\rangle \mid G \text{ is a CFG and } L(G) = \emptyset\}.$$

**Description of a halting TM that accepts $E_{CFG}$**

**Input:** $\langle G\rangle$ where $G$ is a CFG and $L(G) = \emptyset$.

1. Mark all terminals in $G$.
2. For every rule $A \to X_1X_2 \dots X_k$, mark $A$ if all symbols $X_1, X_2, \dots, X_k$ are marked. Repeat this step until no new symbols get marked.
3. If the start symbol is not marked then *accept* else *reject*.

### 16.2.3 CFG Equality Problem

Consider the language,

$$EQ_{CFG} = \{\langle G_1, G_2\rangle \mid G_1, G_2 \text{ are two CFGs and } L(G_1) = L(G_2)\}.$$

One might be tempted to claim that $EQ_{CFG}$ is also decidable but as it turns out it is *not decidable*. We will see a technique to prove this later in the course. For now think about the following questions.

**Exercise 31.** Is $EQ_{CFG}$ Turing recognizable? Is $\overline{EQ_{CFG}}$ Turing recognizable?